

## Rapport de synthèse — FINTRACK Analytics

### 1. Différence entre vue, table et éphémère

- **Vue (view)** : Le modèle est créé comme une vue dans Snowflake. Aucune donnée n'est stockée physiquement ; la requête s'exécute à chaque consultation. C'est idéal pour la couche staging afin d'éviter le stockage superflu.
- **Table (table)** : Les données sont calculées et stockées physiquement dans Snowflake. Cela garantit un temps de réponse rapide pour la couche marts interrogée par les outils BI.
- **Éphémère (ephemeral)** : Aucun objet n'est créé dans Snowflake. Le code SQL est directement injecté sous forme de CTE dans les modèles dbt qui l'appellent, évitant d'encombrer le schéma Snowflake avec des tables de passage. Idéal pour les modèles intermédiaires dans les cas où ce modèle ne participe qu'à la création de 1 (2 max) mart. Si le modèle concerné est central à la création de plusieurs marts, il vaut mieux choisir la matérialisation en Vue.

### 2. Intérêt de la séparation staging, intermediate et marts

- **Staging** : Isole la donnée brute en effectuant le nettoyage de surface (renommage, typage, filtres de base) sans altérer la logique des tables métier.
  - **Intermediate** : Centralise les transformations lourdes, enrichissements et jointures pour éviter la duplication de code SQL.
  - **Marts** : Expose des tables finales agrégées et optimisées pour l'analyse métier.
- Cette découpe modulaire garantit une source unique de vérité, simplifie la maintenance et rend le lineage parfaitement lisible.

### 3. Avantage du test relationships

Le test relationships contrôle **l'intégrité référentielle** entre deux modèles (s'assurer que chaque compte\_id présent dans stg\_transactions existe bien dans stg\_comptes). Ce test permet de rapidement détecter les clés qui sont orphelines et n'ont pas leur correspondants dans la table cible

### 4. Macro 'generate\_schema\_name'

Cette macro permet de redéfinir le nom des schémas cibles lors de la matérialisation. Par défaut DBT nomme les cibles en ajoutant un préfixe (DEV par exemple) avant le véritable nom des schémas cibles pour éviter de perdre des données en production par exemple.

Pour éviter ce fonctionnement par défaut, il faut réécrire cette macro

## **5. Tests et Packages**

DBT offre beaucoup de tests génériques (`not_null`, `accepted_values`, etc) pour valider la qualité des données sources et des modèles. Aussi il y a la possibilité d'intégrer des modules qui proposent des tests beaucoup plus poussés (regex, intervalles de valeurs, etc). J'ai rajouté le module '`dbt_expectations`' dans cette optique pour des tests impossibles à faire avec les tests génériques de DBT.